МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
федеральное государственное автономное образовательное учреждение высшего образования «Санкт-Петербургский политехнический университет Петра Великого»

Институт компьютерных наук и кибербезопасности

Высшая школа технологий и искусственного интеллекта

Направление: 02.03.01 Математика и компьютерные науки

Отчет по практическим работам по дисциплине «Функциональное программирование»

Студент,
5130201/40001 _____ Анари А.

Преподаватель,
к. т. н. _____ Моторин Д. Е.

«_____»_____ 2026 г.

Санкт-Петербург, 2026

Содержание

Введение **3**

1 График выполнения и сдачи работ **4**

2 Описание коммитов **6**

3 Защита практических работ **11**

3.1 Практическая работа 1 11
3.2 Практическая работа 2 11
3.3 Практическая работа 3 11
3.4 Практическая работа 4 11
3.5 Практическая работа 5 12

4 Структура репозитория **13**

Приложения **14**

Введение

Данный отчёт описывает процесс выполнения и защиты практических работ по дисциплине «Функциональное программирование». В ходе семестра было выполнено пять практических заданий. Отчёт отражает хронологию работы, использованные инструменты и результаты защиты.

Общий подход и порядок работы для выполнения практических заданий:

Чтобы подготовиться к каждому заданию, я сначала читал файлы с лекциями и начинал самостоятельно выполнять примеры и задания из них. Я использовал ИИ, чтобы он давал мне простые практические задания после каждого раздела лекций, и выполнял эти задания в отдельных файлах. В основном я пользовался Claude Sonnet 4.6, и мои промпты были примерно такими: «Объясни этот раздел лекции, а затем дай мне 3–5 дополнительных практических заданий по программированию, которые начинаются с лёгкого уровня и постепенно усложняются, чтобы убедиться в понимании материала лекции». Затем я сверял свои ответы с ИИ и задавал ему вопросы, чтобы убедиться, что я понял лекции.

Обычно у меня уходило около 3–6 часов на полное завершение одной лекции, а после каждой лекции я писал практическое задание, прилагаемое к ней, полностью самостоятельно, без использования ИИ. Итак, резюмируя: я использовал ИИ для перевода, понимания и отработки лекционного материала, но не для выполнении практических заданий. После того как я заканчивал с практическими заданиями, вечером я коммитил свои файлы с самостоятельной практикой и официальные практические задания в репозиторий.

Так проходил каждый четверг с самого начала семестра, без единого исключения. Это был мой общий подход и порядок работы при изучении лекций и выполнении практических заданий.

1 График выполнения и сдачи работ

В данном разделе представлены временная диаграмма выполнения и сдачи работ (см. рисунок 1), полный журнал коммитов репозитория (см. таблицу 1), а также список задач (issues) с соответствующими им коммитами (см. таблицу 2).

Таблица 1 содержит полный журнал коммитов с датами и сопоставлением каждого коммита с номером практической работы.

Таблица 2 содержит все задачи (issues), зафиксированные в репозитории, с указанием коммита, закрывшего каждую из них.

Таблица 1: Журнал коммитов репозитория

Хэш	**Дата и время**	**Сообщение коммита**	**ПР**
525a3cd	20.02 15:56	first commit, making repo	—
48da3ba	20.02 16:02	ds store remove	1
8da96d8	20.02 16:03	clean up	1
44416c9	20.02 17:27	first part done	1
271b905	20.02 19:19	finished 1st practice	1
229ce01	20.02 19:20	final change	1
89bc067	01.03 18:28	first part of practice01_2.hs	1
8e9d72d	05.03 18:03	finished practice 1	1
475b4e9	06.03 13:54	fixed myfold1	1
c808add	06.03 14:31	todo 1, wrote test cases for cherrycake and fixed myFoldl1	1
87a6118	06.03 14:43	todo1	1
97e6086	06.03 14:56	final commit lab1	1
4853ff4	08.03 21:25	practice 2 done	2
abf1ef7	12.03 01:16	header cleanup	2
ffa32f2	20.03 01:25	finally done with pr02_2.hs	2
92adb2d	20.03 08:04	clean up	2
6a9a55f	20.03 08:26	added second implementation for MyTree	2
66969fb	20.03 08:42	debugged, added foldable to MyTree	2
7a77be1	22.03 08:47	practiced lecture materials, added practice 9 & 10	2
013e463	22.03 10:40	MyParser now runs perfectly	2
8c93ccf	22.03 10:44	annoying warnings cleared	2
2575b73	22.03 12:46	finished implementing pr2-3	2
16b984e	22.03 12:52	cleanup	2
61a19e6	26.03 19:04	small review	2
bfcb75d	26.03 19:31	fixed an error in MyMaybe that broke Monoid Law	2
26863a4	26.03 19:47	fixed a wrong example	2
87bb47c	26.03 19:54	clean up	2
2fae5a9	26.03 20:14	fixed imports with () which imports NOTHING	2
cbbb2d6	26.03 23:25	cleanup on MyEither	2
4d4ebb5	26.03 23:28	added self practice files	2
7947932	27.03 08:34	clean up	2
a48631c	02.04 12:28	added glitch project for pr3-1	3
1ff58b9	12.04 13:05	solved pr03_2.hs	3
8c1e60c	16.04 19:34	clean up	3
13a6b8b	17.04 23:36	added Monadpr	4
fb00cfe	19.04 16:35	wrote the transformer stack for practice 4	4
3366b62	19.04 17:54	fixing the dependencies on mtl	4

Таблица 1: Журнал коммитов репозитория

Хэш	**Дата и время**	**Сообщение коммита**	**ПР**
7f14dc8	19.04 17:55	add file logging	4
46307e5	19.04 17:56	added a big maze	4
17c6fc2	30.04 15:51	added Practice5 and QCpr	5
5de415f	01.05 18:51	just added pr04_1.hs and pr05_1.hs	4, 5

Таблица 2: Задачи (issues) репозитория и закрывающие коммиты

#	**ПР**	**Название задачи**	**Закрыта**	**Коммит**
1	4	trying hard to understand Monads	17.04 20:36	13a6b8b — added Monadpr
2	4	removing mtl from build dependencies	19.04 14:54	3366b62 — fixing the dependencies on mtl
3	4	wanted to have the logs in a separate file	19.04 15:07	7f14dc8 — add file logging
4	4	real game	19.04 14:56	46307e5 — added a big maze (Fixes #4)
5	5	Maybe a mistake in the book	30.04 12:59	17c6fc2 — added Practice5 and QCpr

На рисунке 1 представлена временная диаграмма выполнения практических работ: коммиты и даты сдачи отмечены разными цветами. Даты сдачи работ:

- ПР1 — 06.03.2026
- ПР2 — 03.04.2026
- ПР3 — 17.04.2026
- ПР4 — 24.04.2026
- ПР5 — 08.05.2026

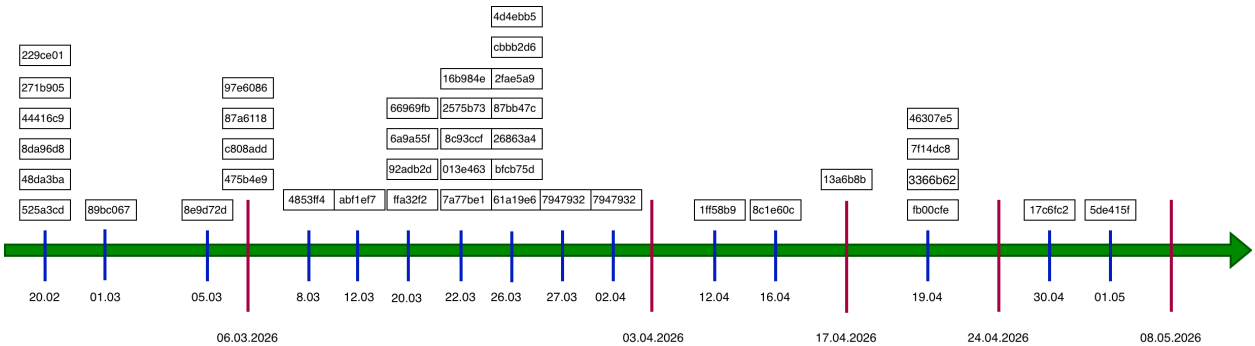

Рис. 1: Временная диаграмма выполнения и сдачи практических работ

2 Описание коммитов

Небольшие коммиты, не требующие отдельного пояснения, объединены в группы. Ряд коммитов описывается отдельно — это крупные коммиты, отражающие работу за несколько дней: как правило, они связаны с изучением объёмных разделов лекций, например монад, где требовалось глубокое погружение в материал.

525a3cd, 48da3ba, 8da96d8 *20.02.2026*

Инициализация рабочего пространства: создание репозитория, настройка `.gitignore` и очистка директории.

- `525a3cd` — создание репозитория, первоначальная структура проекта.
- `48da3ba` — удаление служебных файлов `.DS_Store` из отслеживания.
- `8da96d8` — общая очистка рабочего пространства.

44416c9 *20.02.2026*

Начало выполнения ПР1: реализованы функции первой части `pr01_01`.

- `myFST`, `mySND`, `myTHRD` — извлечение элементов тройного кортежа.
- `myHead`, `myTail` — отделение головы и хвоста списка.
- `myTake`, `myDrop` — взятие и отбрасывание первых n элементов списка.
- `myProduct` — произведение элементов списка.
- `myZip`, `myZip3`, `myUnzip` — объединение и разъединение списков в кортежи.

271b905, 229ce01 *20.02.2026*

Завершение второй части `pr01_01`: реализованы оставшиеся функции высшего порядка.

- `myFilter`, `myFilter2` — фильтрация списка (два варианта: через охранные выражения и через `if-else`).
- `myMap` — применение функции к каждому элементу списка.
- `myzipWith`, `myZipWith3` — попарное и потройное применение функции к спискам.
- `myAll`, `myAll2`, `myAny`, `myAny2` — проверка предиката на всех/хотя бы одном элементе (два варианта каждая).
- `myComposition` — композиция двух функций.

89bc067 *01.03.2026*

Начало выполнения ПР1, часть 2: создан файл `pr01_2.hs`. Реализованы функции на основе сверток (`myReverse`, `myTakeWhile`, `mySpan`), свёртки для непустых списков (`myFoldl1`, `myFoldr1`), работа с неравными списками (`myZipSave`, `myUnzipSave`), а также `myMaybe`, `myMap` для пользовательского типа `MyList` и развёртка `myUnfoldr`.

8e9d72d *05.03.2026*

Завершение ПР1: дописана вторая часть `pr01_2.hs`. Реализованы алгебраические типы данных для выпекания тортов

475b4e9, c808add, 87a6118, 97e6086 *06.03.2026*

Исправления по итогам защиты ПР1: устранены замечания, полученные в ходе защиты, — исправлена `myFoldl1` и написаны тестовые случаи для функций выпекания тортов. Вопросы, заданные в ходе защиты, и их решения приведены в разделе 3.

4853ff4, abf1ef7 *08.03 — 12.03.2026*

Выполнена ПР2, часть 1: создан cabal-проект `MyEvolProject`, добавлен модуль `MyEvolModule.hs` с реализацией инстансов классов типов, проект собран и протестирован. Коммит `abf1ef7` — небольшая правка заголовков.

ffa32f2, 92adb2d *20.03.2026*

Крупный коммит по ПР2: предварительно отработан весь лекционный материал в файлах самостоятельной практики (`practice2.hs`–`practice8.hs` в директории `self practice`), затем выполнена ПР2, часть 2 — создан stack-проект `myParser` с модулями `MyMaybe`, `MyEither`, `MyTree`, написаны и прогнаны тестовые случаи.

6a9a55f *20.03.2026*

В `MyTree.hs` добавлена вторая реализация дерева, в которой операция применяется к каждому листу относительно всего дерева.

66969fb *20.03.2026*

Отладка проекта: добавлен инстанс `Foldable` для `MyTree`, обновлён `Main.hs` с тестовыми вызовами, скорректированы зависимости в `package.yaml` и `.cabal`.

7a77be1 *22.03.2026*

Самостоятельная практика по лекционному материалу до раздела `Alternative`: добавлены файлы `practice9.hs` и `practice10.hs`. Отработаны: пользовательский тип `Parser`, инстансы `Functor`, `Applicative` и `Alternative` для него, примитивные парсеры (`satisfy`, `char`, `digit`, `lower`, `upper`), комбинаторы `some`/`many`, построение парсеров выражений (`finalMult`, `plusOrMult`).

013e463, 8c93ccf *22.03.2026*

Реализован модуль `MyParser.hs` в рамках ПР2, часть 3: парсер построен поверх собственного типа `MyMaybe` вместо стандартного `Maybe`. Реализованы инстансы `Functor`, `Applicative`, `Alternative`, примитивные парсеры и комбинаторы (`lowers`, `digits`, `finalMult`, `plusOrMult`). Обновлён `Main.hs`, созданы заглушки для `MyParsec` и `MyAttoparsec`. Коммит `8c93ccf` — устранены предупреждения компилятора.

2575b73, 16b984e *22.03.2026*

Завершена ПР2, часть 3: реализованы `MyParsec.hs` и `MyAttoparsec.hs`. `MyParsec.hs` использует готовые функции из `Text.Parsec` с написанными обёртками для сохранения интерфейса `Main.hs`. `MyAttoparsec.hs` использует функции из `Data.Attoparsec.Text` с применением `pack` из `Data.Text` для передачи строки в `parseOnly`. Также добавлены файлы самостоятельной практики `practice11.hs` и `practice12.hs`. Коммит `16b984e` — финальная очистка.

61a19e6, bfcb75d, 26863a4, 87bb47c, 2fae5a9, cbbb2d6 *26.03.2026*

Серия мелких исправлений по проекту: исправлена ошибка в `MyMaybe`, нарушавшая закон моноида; исправлен некорректный пример, оставшийся с предыдущей версии; устранена ошибка с импортом вида `import M ()` (импортирует ничего); очистка модуля `MyEither`.

4d4ebb5, 7947932 *26.03 — 27.03.2026*

Добавлены файлы самостоятельной практики поздно ночью; в сообщении коммита отмечено, что часть содержимого `practice12.hs` была случайно перезаписана от усталости. `practice12.hs` — практика по классам типов: пользовательский тип `Direction` с инстансами `Show`, `Eq`, `Enum`, `Bounded`; `Foldable` и `Monoid` для `MyEither` и `MyMaybe`. `practice13.hs` — повторная отработка реализации `Parser` поверх `MyMaybe` с инстансами `Functor`, `Applicative`, `Alternative` и парсерами-комбинаторами.

a48631c *02.04.2026*

Крупный коммит: создан stack-проект `glitches` для ПР3, часть 1, добавлены файлы самостоятельной практики `practice14` и `practice15`. Файлы `practice14` и `practice15` — самостоятельная практика по материалам книги *Get Programming with Haskell* Уилла Курта, посвящённой теме глитч-арта.

Проект реализует глитч-арт на изображениях через манипуляцию байтами (`Data.ByteString`). Модуль `Purefuncs.hs` содержит чистые функции: `replaceByte` — замена одного байта по по-

зиции, `sortSection` — сортировка участка байтов с реверсом. `Main.hs` реализует три режима глитча: случайная замена байта, случайная сортировка участка, и их комбинация через `foldM` (вместо цепочки ручных вызовов, как предлагалось в книге) — применяется список из 3 замен и 2 сортировок. Результаты записываются в отдельные файлы; в репозиторий включены выходные изображения (`lovecraft.jpg` и тестовые снимки).

1ff58b9, 8c1e60c *12.04 — 16.04.2026*

Выполнена ПР3, часть 2: создан stack-проект `Route` с реализацией поиска пути в лабиринте через монадный трансформер `RWS`. `MazeSolver.hs` определяет тип `Maze` (`Map String [String]`), монадный стек `Solver` (`RWS Maze [String] (Set String)`), и реализует DFS через взаимно-рекурсивные функции `dfs` и `tryNeighbors` — по аналогии с `some`/`many`. `Main.hs` запускает решатель через `runRWS` на тестовом лабиринте и выводит лог посещений и найденный путь. Также предпринята попытка применить глитч-эффекты к `.bmp`-формату — без успеха; соответствующие файлы включены в коммит.

До выполнения этого задания были отработаны монады `Reader`, `Writer` и `State` в файлах самостоятельной практики, однако они не были закоммичены здесь — все файлы практики по монадам включены в следующий коммит.

13a6b8b *17.04.2026*

Самый крупный коммит в репозитории, закрывает задачу #1. Все файлы самостоятельной практики по монадам и монадным трансформерам, накопленные за несколько дней, зафиксированы в одном коммите в директории `self practice/MonadPr/`. Материал изучался по двум источникам: книга *Get Programming with Haskell* Уилла Курта (главы 30–32) и *Learn Haskell by Example* (глава 15, монадные трансформеры).

Содержимое директории `MonadPr/`:

- `MONADNOTES.hs` — конспект-шпаргалка по API монад `Reader`, `Writer`, `State`: ключевые функции, сигнатуры и примеры для повторения.
- `MReaderpr.hs` — практика монады `Reader`: `ask`, `asks`, `local`, цепочки вычислений в контексте окружения.
- `MWriterpr.hs` — практика монады `Writer`: `tell`, `listen`, `censor`, накопление лога.
- `MStatepr.hs` — практика монады `State`: `get`, `put`, `modify`, цепочки состояний через `replicateM`.
- `MTrpr.hs` — практика монадных трансформеров: стеки `ReaderT`/`WriterT`/`StateT`, `lift`, `runReaderT`/`runWriterT`/`runStateT`.
- `MPTCpr.hs` — практика `MultiParamTypeClasses`: класс `Mult` для умножения векторов и матриц.
- `MRWSpr.hs` — практика монады `RWS` как комбинации `Reader`/`Writer`/`State`.
- `Maze/` — самостоятельный stack-проект: повторная реализация лабиринта через `RWS` для закрепления материала перед сдачей ПР3.

Содержимое директории `getprgwithHaskell/`:

- `chapter30.hs` — глава 30: монада `IO`, do-нотация, `>>=` и `>>`.
- `chapter31.hs` — глава 31: работа с `IO` и модулями, модуль `Pizza`.
- `chapter32.hs` — глава 32: монада списков, `guard`, списочные включения через `do`.
- `pizza.hs` — упражнение из книги: интерактивная программа для сравнения цен на пиццу через `IO`.
- `review.hs` — упражнение на повторение: комбинирование `Maybe`, `Applicative` и функторов.

- `practice16.hs` — дополнительная самостоятельная практика по материалу глав.

Лекционный материал по монадам (лекции 9, 10, 11) отработан в отдельных stack-проектах, по одному файлу на каждый тип монады:

- `lec09/`: `MonadMaybe`, `MonadList`, `MonadReader`, `MonadWriter`, `MonadId`, `MonadExcept`.
- `lec10/`: `MonadState`, `MonadRWS`, `MonadIO`, `MonadLogic`, `MonadPar`.
- `lec11/`: `TrMon` — трансформеры монад; `TrRW` — трансформеры `Reader`/`Writer`; `MPTC` — многопараметрические классы типов.

Стоит отметить, что изучение монад и привыкание к ним с той скоростью, с которой шли лекции, было настоящим испытанием. Я чувствую, что действительно выложился по полной в этой теме — за те две недели, что я практиковался с монадами, было много ночей, когда я засиживался допоздна. Мне кажется, мне удалось их освоить благодаря использованию упомянутых книг и источников. Стоит сказать, что монады стали одной из самых запоминающихся частей моего обучения на данный момент!

fb00cfe *19.04.2026*

Начало ПР4: создан stack-проект `MazeGame`, реализован модуль `MyTransformerStack.hs`. Модуль содержит реализацию трёх монадных трансформеров с нуля: `ReaderT` (с инстансами `Functor`/`Applicative`/`Monad`/`MonadTrans` и функциями `ask`, `asks`, `local`), `WriterT` (с `tell`, `listen`, `censor`) и `MyStateT` (с `get`, `put`, `modify`). Класс `MonadTrans` также определён вручную. Реализации перенесены из файлов самостоятельной практики и адаптированы под нужды проекта. Файл `GameEngine.hs` создан пустым — работа над ним запланирована в следующих коммитах.

3366b62 *19.04.2026* (закрывает задачу #2)

Реализован `GameEngine.hs` и исправлены зависимости. Задача #2 была связана с тем, что `mtl` использовался только ради `runIdentity` — он заменён на `Data.Functor.Identity` из `base`, и `mtl` полностью убран из зависимостей, что соответствует цели задания: трансформерный стек написан вручную. `GameEngine.hs` определяет монадный стек `Game a = MyStateT RoomName (WriterT [String] (ReaderT Maze IO)) a`, функции игрового цикла (`gameTurn`, `gameLoop`), парсер лабиринта из файла (`parseMaze`, `loadMaze`) и вспомогательную функцию `io` для подъёма `IO`-действий через три слоя трансформеров. Добавлен тестовый файл лабиринта `maze.txt`.

7f14dc8, 46307e5 *19.04.2026* (закрывают задачи #3 и #4)

Два завершающих коммита ПР4. `7f14dc8` (задача #3) — добавлен файл `logs.txt` с сохранёнными логами игровых сессий, реализованных через функцию `saveLogs` в `GameEngine.hs`. `46307e5` (задача #4) — добавлен большой лабиринт `maze1.txt` для полноценной игры: 22 комнаты с разветвлённой структурой (главный зал, библиотека, арсенал, подземелье, деревня, таверна, крипта и т.д.) с единственным путём до финала через `throne -> finish`.

17c6fc2 *30.04.2026* (закрывает задачу #5)

Крупный коммит: выполнена ПР5, добавлена самостоятельная практика по QuickCheck в директории `self practice/QCpr/`.

Подготовка к заданию: лекционный материал изучен полностью, просмотрено видео о `classify` и `collect`, прочитаны главы 36 и 37 книги *Get Programming with Haskell* Уилла Курта. В процессе изучения была обнаружена ошибка в коде из книги (задача #5): функция `isPrime` использует охранное выражение `n >= length primes` в качестве верхней границы, что логически неверно. При `primes = sieve [2..20]` длина списка равна 8, поэтому `isPrime 19` возвращает `Nothing`, хотя 19 является простым числом. Исправление: заменить `length primes` на `head . reverse $ primes`, т.е. сравнивать с наибольшим элементом списка, а не с его длиной — именно так реализовано в `QCpr/primes/src/Primes.hs`.

Реализация ПР5 (проект `practice5/prac5/`):

- `src/Lib.hs` — реализованы функции `addMod` и `reverseWords`; здесь же находятся черновые варианты всех property-тестов с подробными комментариями и результатами запусков `quickCheck` — оставлены намеренно как рабочие заметки.
- `test/Spec.hs` — финальная чистовая версия: 7 property-тестов с корректными именами (`prop_addModMatchesModuloAddition`, `prop_addModHasNeutralElement`, `prop_addModIsCommutative`, `prop_reverseWordsEmptyString`, `prop_reverseWordsSingleWord`, `prop_reverseWordsChangesWordOrd` `prop_reverseWordsTwiceReturnsOriginal`).

Содержимое директории `self practice/QCpr/`:

- `palindrome-testing/` — практика по примеру из книги: тестирование функции `isPalindrome` через QuickCheck.
- `primes/` — практика тестирования `isPrime` и `primeFactors` с исправлением ошибки из книги; реализованы тесты `prop_validPrimesOnly`, `prop_primesArePrime`, `prop_nonPrimesAreComposi` `prop_factorsMakeOriginal`, `prop_allFactorsPrime`.
- `prac5/` — черновой вариант проекта ПР5, использовался для экспериментов перед написанием финальной версии.
- `pr05_1.hs`, `notes.hs` — вспомогательные файлы с заметками.

3 Защита практических работ

3.1 Практическая работа 1

Вопрос 1. Написать тест для функции `makeCake`.

Ответ:

```
testmake = makeCake (Just CherryDough) (Just (Bake 16))

```

Вопрос 2. Написать функцию с типом `funC :: Either a b -> (a -> b) -> b` и тестовый случай.

Ответ:

```
funC :: Either a b -> (a -> b) -> b
funC (Left a)  f = f a
funC (Right b) f = b

testF = funC (Right 3) (+1)

```

3.2 Практическая работа 2

Устные вопросы.

Вопрос: Что такое полугруппа (Semigroup)?

Ответ: Тип с ассоциативной бинарной операцией `(<>)`. Единственный закон: `(x <> y) <> z = x <> (y <> z)`.

Вопрос: Что такое моноид (Monoid)?

Ответ: Полугруппа с нейтральным элементом `mempty`. Дополнительные законы: `mempty <> x = x` и `x <> mempty = x`.

Вопрос 1. Каков тип выражения `a = ((\x y z -> x + y + z) <$> Right 3)`?

Ответ: `Either a (Int -> Int -> Int)`

Вопрос 2. Заполнить пропуски так, чтобы выражение `a = (... <*> ... <*> ...)` имело тип `Maybe String`.

Ответ:

```
a = Just (:) <*> Just 'x' <*> Just "ab"

```

3.3 Практическая работа 3

Вопрос. Написать пример монадического связывания `(>>= )` для монады `Writer`.

Ответ:

```
w = writer ("a", [1,2]) >>= \x -> writer (x ++ "b", [3,4])

```

3.4 Практическая работа 4

Устный вопрос. Почему `runGame` имеет тип `Game () -> Maze -> IO ()`, если `Game a` параметризован типом `a`?

Ответ: Тип `Game a` параметризован, чтобы позволить различным вычислениям возвращать разные типы — например, `gameTurn :: Game Bool` возвращает `Bool`, `getNeighbors :: Game [RoomName]` возвращает список комнат. В случае `runGame` передаётся `Game ()`, потому что `gameLoop :: Game ()` не возвращает содержательного значения — он существует только ради эффектов: изменения состояния (`StateT`), накопления лога (`WriterT`) и ввода-вывода

(`IO`). Тип `()` — стандартный способ выразить «нет возвращаемого значения» в Haskell, аналогично `void` в других языках. При раскрутке стека `runMyStateT`, `runWriterT`, `runReaderT` все обёртки снимаются и в итоге остаётся `IO ()`.

3.5 Практическая работа 5

4 Структура репозитория

Репозиторий доступен по адресу: https://github.com/Ali-sbp/Anari-FP-Practice. Корневая директория содержит по одной папке на каждую практическую работу, а также директорию самостоятельной практики.

- `practice 1/` — файлы ПР1.
  - `pr01_1.hs` — функции для работы со списками и кортежами.
  - `pr01_2.hs` — свёртки, развёртки, алгебраические типы данных, выпечка тортов.
- `practice2/` — файлы ПР2.
  - `pr02_1.hs` — классы типов, инстансы.
  - `pr02_2.hs` — деревья, `MyMaybe`, `MyEither`.
  - `pr02_3.hs` — вспомогательный файл.
  - `MyEvolProject/` — cabal-проект: модуль `MyEvolModule.hs` с инстансами классов типов.
  - `myParser/` — stack-проект: `src/MyTypes/` (`MyMaybe`, `MyEither`, `MyTree`), `src/MyParsers/` (`MyParser`, `MyParsec`, `MyAttoparsec`).
- `practice3/` — файлы ПР3.
  - `pr03_1.hs`, `pr03_2.hs` — основные файлы задания.
  - `glitches/` — stack-проект: глитч-арт через манипуляцию байтами (`src/Purefuncs.hs`, `app/Main.hs`).
  - `Route/` — stack-проект: поиск пути в лабиринте через `RWS` (`src/MazeSolver.hs`).
- `practice4/` — файлы ПР4.
  - `pr04_1.hs` — основной файл задания.
  - `MazeGame/` — stack-проект: игра в лабиринт на трансформерном стеке. `src/MyTransformerStack` — ручная реализация `ReaderT`/`WriterT`/`StateT`; `src/GameEngine.hs` — игровая логика; `maze.txt`, `maze1.txt` — файлы лабиринтов.
- `practice5/` — файлы ПР5.
  - `pr05_1.hs` — основной файл задания.
  - `prac5/` — stack-проект: `src/Lib.hs` — функции и черновые тесты с заметками; `test/Spec.hs` — финальные property-тесты на QuickCheck.
- `self practice/` — вся самостоятельная практика.
  - `practice2.hs` – `practice13.hs` — последовательные файлы практики по лекционному материалу (парсеры, классы типов, `Alternative` и др.).
  - `practice14`, `practice15` — скомпилированные программы глитч-арта из книги *Get Programming with Haskell*.
  - `MonadPr/` — практика по монадам и трансформерам: отдельные файлы (`MReaderpr.hs`, `MWriterpr.hs`, `MStatepr.hs`, `MTrpr.hs`, `MPTCpr.hs`, `MRWSpr.hs`, `MONADNOTES.hs`), stack-проекты по лекциям 9–11 (`lec09/`, `lec10/`, `lec11/`), материалы книги (`getprgwithHaskell/`) и практический лабиринт (`Maze/`).
  - `QCpr/` — практика по QuickCheck: `palindrome-testing/`, `primes/` (с исправлением ошибки из книги), `prac5/` (черновик ПР5).

Приложения